Blockchain-Enhanced Data Sharing With Traceable and Direct Revocation in IIoT

Keping Yu, *Member, IEEE*, Liang Tan, Moayad Aloqaily, *Member, IEEE*, Hekun Yang, and Yaser Jararweh, *Member, IEEE*

***Abstract*—The industrial Internet of Things (IIoT) supports recent developments in data management and information services, as well as services for smart factories. Nowadays, many mature IIoT cloud platforms are available to serve smart factories. However, due to the semicredibility nature of the IIoT cloud platforms, how to achieve secure storage, access control, information update and deletion for smart factory data, as well as the tracking and revocation of malicious users has become an urgent problem. To solve these problems, in this article, a blockchain-enhanced security access control scheme that supports traceability and revocability has been proposed in IIoT for smart factories. The blockchain first performs unified identity authentication, and stores all public keys, user attribute sets, and revocation list. The system administrator then generates system parameters and issues private keys to users. The domain administrator is responsible for formulating domain security and privacy-protection policies, and performing encryption operations. If the attributes meet the access policies and the user's ID is not in the revocation list, they can obtain the intermediate decryption parameters from the edge/cloud servers. Malicious users can be tracked and revoked during all stages if needed, which ensures the system security under the *Decisional Bilinear Diffie–Hellman* (DBDH) assumption and can resist multiple attacks. The evaluation has shown that the size of the public/private keys is smaller compared to other schemes, and the overhead time is less for public key generation, data encryption, and data decryption stages.**

Manuscript received July 29, 2020; revised October 28, 2020 and December 5, 2020; accepted December 15, 2020. Date of publication January 5, 2021; date of current version July 26, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 61373162, in part by Sichuan Provincial Science and Technology Department Project under Grant 2019YFG0183, and in part by the Japan Society for the Promotion of Science (JSPS) Grants-in-Aid for Scientific Research (KAKENHI) under Grant JP18K18044. Paper no. TII-20-3659. *(Corresponding author: Liang Tan.)*

Keping Yu is with the College of Computer Science, Sichuan Normal University, Chengdu 610101, China, and also with the Global Information and Telecommunication Institute, Waseda University, Tokyo 169-8050, Japan (e-mail: keping.yu@aoni.waseda.jp).

Liang Tan is with the College of Computer Science, Sichuan Normal University, Chengdu 610101, China, and also with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China (e-mail: jkxy_tl@sicnu.edu.cn).

Moayad Aloqaily is with Qatar University, 2713 Doha, Qatar (e-mail: maloqaily@ieee.org).

Hekun Yang is with the College of Computer Science, Sichuan Normal University, Chengdu 610101, China (e-mail: 1014191945@qq.com).

Yaser Jararweh is with the Jordan University of Science and Technology, Irbid 3030, Jordan (e-mail: yijararweh@just.edu.jo).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2021.3049141.

Digital Object Identifier 10.1109/TII.2021.3049141

***Index Terms*—Big data, blockchain, data sharing, industrial Internet-of-Thing (IIoT), smart factory, traceable and revocation.**

I. Introduction

THE INDUSTRIAL Internet of Things (IIoT) aims to realize intelligent industry through the application of intelligent terminals with sensing capabilities, ubiquitous mobile computing patterns, and pervasive mobile network communication to all aspects of industrial production [1]. IIoT as well as green IoT is designed to collect large amounts of data, and then use that data to troubleshoot, identify performance bottlenecks, and detect malicious behavior [2]. Therefore, the realization of a smart factory with IIoT as the core is crucial to the development of smart manufacturing [3]. Smart factory is a new stage in the development of information technology for modern factories. It uses IIoT technology to enhance information management and services [4]. In smart factories, a wide range of sensors in each domain and each application are used to collect data. With the rapid growth of advanced IIoT technology, processing, storage, light-weight security protocols, and data acquisition are increasingly more easy and convenient for smart factories. This supports the functionality and value of industrial big data, known as smart factory big data (SFBD) [5]. SFBD has been widely studied due to its huge application value to workers, managers, and decision-makers. In particular, how SFBD can be securely shared and privacy protected on the IIoT cloud platform becomes particularly important [6].

Currently, there are many mature IIoT clouds platforms for smart factories, such as Siemens mindsphere and GE predix [7]. These IIoT cloud platforms enable the integration of various devices in a smart factory, thus, solving the information island problem between devices. IIoT cloud platform has become the future development trend of smart factory in terms of data management and information sharing services. However, as a semitrusted third party, the IIoT cloud platform has a great security risk when managing factory information. Management privileges can be abused and leak factory private information. To solve the problems of data confidentiality and integrity, many data encryption technologies have been applied to the security of cloud outsourcing data. Ciphertext or attribute-based encryption are just examples of access control technologies for an untrusted cloud storage server [8]. Related Cryptographic schemes have been proposed earlier to achieve confidentiality and protect

1551-3203 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

privacy. However, there are still many issues, particularly with respect to malicious user tracking and revocation.

In 2008, digital currency (*aka* Bitcoin), which is based on blockchain, has emerged, and since then, applying blockchain technology to the IIoT became feasible and viable. Blockchain is a distributed, traceable and nontamperable ledger. Its nodes share data and jointly maintain the ledger, as well as use consensus mechanisms to ensure data consistency [9]. In addition, the blockchain is also time-ordered. Thus, blockchain outperforms today's commonly used clouds and databases in terms of tamper-proofing, privacy protection, and temporal ordering [10]. As an emerging security technology, blockchain is not yet deeply integrated with the production, management, operations, and sales of smart factories. In smart factories, some data must be shared across the entire production range, such as inventory data, product investment, fund recovery, etc. These data are time-sequential and need to be traced throughout the entire process. At the same time, it needs to be tamper-proof, nonduplicative, and traceable. Blockchain has just the right characteristics to meet these needs of a smart factory [11].

In this article, we proposed a blockchain-enhanced security access control scheme that supports traceability and revocability in IIoT for smart factories. Our main contributions are as follows.

First, we proposed an attribute-based access control scheme for the smart factory that supports traceability and revocation. It proved to be safe under the DBDH assumption. Compared with other schemes, the size of the public key and the private key is shorter, and the time overhead is smaller in the public key generation, data encryption, and data decryption stages.

Second, we employed the blockchain to perform unified identity authentication and are responsible for storing all public keys, user attribute sets, and suspension lists.

The rest of this article is organized as follows. In Section II, the related literature is reviewed. In Section III, we discuss the system architecture. Section IV discusses protocol design, and Section V focuses on scheme analysis. Section VI is a simulation experiment. Finally, Section VII concludes this article.

II. Related Work

With the development of IIoT, the smart factory has become the focus of attention. Secure access to industrial information is key to smart factory research and will be mainly based on emerging technologies such as AI and ML [12]. The related research is roughly divided into three categories. The first category refers to the security of the IIoT and smart factories, including industrial equipment and network security. The second category addresses the cloud security that ensures data storage. The third category covers the integration of IIoT and blockchain to ensure industrial security [13].

In terms of IIoT and smart factory security, an intelligent control secure model of fog nodes in healthcare via the use of cognitive fog nodes has shown good application to smart health factories [14]. Jung *et al.* [15] introduced an IIoT concept-based smart factory web. And to meet the real-time needs of the IIoT, Shin *et al.* [16] combined superintelligent computing technology related to collecting information using ICT technology in various industries and proposed a network topology based on edge/fog computing. Domova and Dagnino [17] focused on alarm processing in smart factories and the proposed set of data mining alarm algorithms. In order to promote intelligent production, data exchange, and management of equipment in smart factories and industrial warehouses, and to establish and maintain a reliable cloud platform, Huh *et al.* [18] proposed building the reindustrial architecture.

With the need to secure and protect industrial data, new control models are being investigated. BLP-based, task-based, UCON-based are all examples of access control models. Tan *et al.* [19] have proposed an access control framework that for smart devices in green IoT is based on blockchain technology. Moreover, Yang *et al.* [20] have designed a multitenant access control system, which relies on the role of the tenant. *CloudPolice* is another multitenant access control model-based on hypervisor [21]. Furthermore, access control schemes based on attribute-encryption have been shown in [8], [22], and [23].

Ostrovsky *et al.* [22] expand the attribute space on the basis of Bethencourt *et al.* [24], and propose a "one-to-many" outsourcing encryption scheme that supports a monotonic access control structure. In order to solve the problem of revoking and tracing malicious users in the cloud environment, an ABE solution that supports proxy re-encryption is proposed [13]. Yan and Meng [25] propose an access control scheme that supports direct revocation. In the encryption phase, the revocation list is embedded in the ciphertext. However, this scheme cannot deal with the updating problem of the access policy, and [23] also has similar problems. In [13], [22], [23], and [25], the encryptor and decryptor need to bear a lot of computational overhead. Nguyen *et al.* [26] proposed a fine-grained attribute access control scheme for IoT that supports outsourcing. However, in the decryption process, it still has a large overhead. Ma *et al.* [27] proposed an encryption scheme that supports both encryption and decryption outsourcing based on the different characteristics of public clouds and private clouds. It also can verify the correctness of the outsourcing results, but this scheme has poor scalability. With the help of the fog computing environment, Zhang *et al.* [28] proposed a scheme to support encryption, decryption, and attribute revocation, which can save the computing and storage costs of IoT devices. However, this scheme needs to hand over the symmetric key to the fog computing node during the encryption process, which is a very unsafe behavior. The detailed comparison of the abovementioned work is listed in Table I.

Combining blockchain with IIoT has become popular, using it as a trusted entity to ensure the security and credibility of IIoT and smart factories. Wan *et al.* [6] combined traditional IIoT architecture and blockchain to construct and reshape a distributed network. Huang *et al.* [29] combined the flexibility and security capabilities of blockchain to create a consensus-based credit mechanism and credit-based proof-of-work instrument to deal with IIoT devices that are unsuitable for power consumption due to blockchain's extensive power consumption and low throughput. Lee *et al.*, [30] proposed the idea of combining Ethereum and deep reinforcement machine learning with blockchain to create a reliable and secure environment for data collection

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF RELATED WORKS

Solution	Access structure	Outsourcing encryption	Agency re-encryption	Outsourcing decryption	Traceability	Security
[13]	AND-gates	N	Y	N	Y	DBDH DLP
[22]	LSSS	N	N	N	N	DBDH
[23]	LSSS	N	N	N	N	q-BDH
[25]	Access tree	N	N	N	N	DBDH
[26]	Access tree	Y	N	N	N	DLP
[27]	LSSS	Y	N	Y	N	DLP
[28]	Access tree	Y	Y	Y	N	DBDH

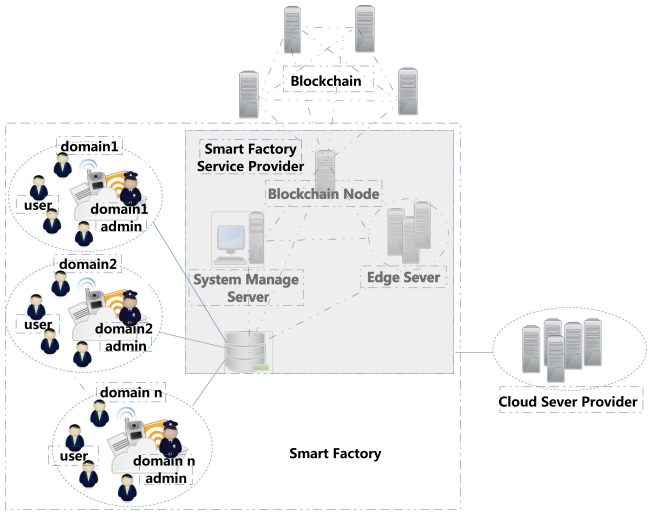


Fig. 1. System architecture.

and sharing. Liu *et al.*, [31] proposed an industrial IoT fog system architecture based on blockchain. Jang *et al.*, [32] used blockchain technology to ensure security of participants' control of resources when interacting with different smart factories. Teslya *et al.* [33] proposed a lightweight IIoT platform system based on blockchain to solve security problems in the construction process of IIoT ecosystems.

III. SYSTEM ARCHITECTURE

A. System Architecture

The framework of the proposed secure and privacy-preserving SFBD sharing system is shown in Fig. 1. There are five entities in the system: Domain, Blockchain, System Manage Server, Edge Sever, and Cloud Sever Provider.

1) *Domain*. Domains are the basic management units of a smart factory and include users, devices, sensors, and domain administrators. Generally, each domain has one administrator and many devices and sensors, and generates large amounts (of all types) of data that make up the SFBD. Depending on the smart factory's access control policy, users and administrators can access not only their own domain's data but also data from other domains. The domain administrator is the workshop director or the person in charge of the basic functional unit of the smart factory, and is responsible for setting the domain security and privacy protection policy, performing encryption operations. In this article, we assume that this is a trusted role.
2) *Blockchain*. Blockchain performs unified identity authentication for smart factory, and all users must register on blockchain. Blockchain stores public keys of a system manager, domain manager and users, as well as traceable lists. The blockchain in this article is an alliance chain.
3) *System Management Sever (SMS)*. SMS is a factory management organization that provides smart factory services to users. SMS generates public and master keys for the entire system, and private keys and cert for users. It is as a trusted entity.
4) *Edge Server (ES)*. ES stores encrypted factory temporary data to enable policy matching and provide data services to users. If policy matching is successful, decryption parameters are generated and published. Otherwise, it returns nothing. It is trusted in this system.
5) *Cloud Service Provider (CSP)*. CSP is a semitrusted entity. It stores factory historical encrypted data, enables policy matching, and provides data services to users. If policy matching is successful, it produces and publishes the decryption parameters. Otherwise, it returns nothing.

B. Security Model

In our security model, a revoked user can illegally access data by colluding with a user in the same domain. Suppose that the revoked user can obtain a private key that satisfies a specific access structure, and that this technique is not relayed to the compromised domain. Instead, an existing user can obtain a private key that does not satisfy a specific access structure but is the current version of the private key. To more accurately describe the security model, we define a game between adversary A and challenger B as follows [34]. Three types of adversary are defined as below.

1) *Type-I:* The revoked user obtains a private key that satisfies a specific access structure, which does not include the current version of the attacked domain.
2) *Type-II:* Obtaining the private keys which is the current version and do not satisfy the specific access structure by the existing users.
3) *Type-III:* Obtaining the private keys which is the current version and do not satisfy the specific access structure by the revoked users excluding the attacked domain.

However, Type-III is the weakening of Type-I, so we just need to construct Type-I query and Type-II query. Notations used throughout this article are listed in Table II.

Init A selects T^*, Domain* and V^*, and sends them to B.

Setup B first calls Setup() to get MK and PK, and then B calls DomainSetup() to get DMK_0 and DPK_0 for Domain*. Then B calls DomainUpdate() to get DMK_V,

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PARAMETERS DEFINITION

Description	Notation	Description	Notation
Security parameter	λ	User set	A
Public key and system master key	PK, MK	Global attributes set	U
Registration list	W	Access policy	(ρ, M)
Attributes set and identification of i	L_{ui}, ID_{ui}	ciphertext	CT
Access structure	T, T^*	Private key	SK_{ui}
The list of revoked users	R, R^*	plaintext	m
Intermediate parameter	DK_{ui}	User's Cert	$Cert$

DPK_V, and $Re-\text{Key}_{V-1\to V}$ from $V=1$ to V^*. Finally, B sends PK and $\{\text{DPK}_V\}_{0\le V\le V^*}$ to adversary A. B keeps $\{\text{Re-Key}_{V\to V+1}\}_{0\le V\le V^*-1}$, MK and $\{\text{DMK}_V\}_{0\le V\le V^*}$.

Phase 1: A initiates some queries. In this process, the abilities of the revoked users and the existing users are, respectively, formalized by Type-I query and Type-II query.

1) *Type-I query:* Private key query $<\text{ID}, \text{Domain}^*, S>$, where S (the attribute set) satisfies T^* (the access policy) but V (the version) must satisfies $V<V^*$. B calls $\text{KeyGen}(\text{PK}, \text{MK}, \text{DPK}_V, S, Domain^*)$ to create the private key SK_V. B returns SK_V to A.
2) *Type-II query:* Private key query $<\text{ID}, \text{Domain}^*, S>$ where S does not satisfy T^*. B runs $\text{KeyGen}(\text{PK}, \text{MK}, \text{DPK}_{V^*}, S, \text{ID})$ to create SK_{V^*}. B returns SK_{V^*} to A.
3) *User update query:* $<\text{ID}, \text{SK}_V>$ where ID has existed in Type-I query and $V<V^*-1$. B returns with UP_{V+1} to A. A calls $\text{UserUpdate}(\text{DSK}_{\text{ver}}, UP_{V+1})$ to obtain the key DSK_{V+1} (updated).
4) *Re-encryption query:* $<\text{CT}_V>$ where CT_V and $V<V^*$ is the ciphertext. B returns with CT_{V+1} through running $\text{Re-Encrypt}(\text{CT}_V, \text{Re}-\text{Key}_{V\to V+1})$.

Challenge: A commits two plaintexts M_0 and M_1 with equal length to B once A checks that *Phase 1* ends. B randomly selects $\tilde{b}\in\{0,1\}$ and computes $\text{CT}_{\tilde{b}}=\text{Encrypt}(\text{PK}, \text{DPK}^*, M_{\tilde{b},A^*})$ then returns $\text{CT}_{\tilde{b}}$ to A.

Phase 2. A initiates other queries as in *Phase 1*, and B answers as in *Phase 1*.

Guess: A exports a guess $\tilde{b}'\in\{0,1\}$. If $\tilde{b}=\tilde{b}'$, then he wins the game. The advantage is defined as $\text{Adv}_A=|Pr[\tilde{b}=\tilde{b}']-\frac{1}{2}|$.

IV. PROTOCOL DESIGN

A. Overview of the Scheme

To achieve a secure and traceable SFDB sharing system with cloud/ES storage, First, we define functions as follows.

1) $\text{IDGen}(\text{Cert}_{ui})\to\text{addID}$: Inputs user's password to blockchain, returns an account address addrID.
2) $\text{setup}(\lambda)\to(\text{PK}, \text{MK})$: Inputs security parameter λ, and outputs system's PK and MK.
3) $\text{KeyGen}(\text{MK}, L_{ui}, \text{ID}_{ui})\to(\text{Cert}_{ui}, \text{SK}_{ui})$: Inputs MK, attributes set L_{ui} and user identification ID_{ui}, and returns a private secret key SK_{ui}, a user cert Cert_{ui}.
4) $\text{Encrypt}(\text{PK}, (\rho, M), m)\to C_{pi}$: Inputs PK, (ρ, M), and m returns a ciphertext CT.

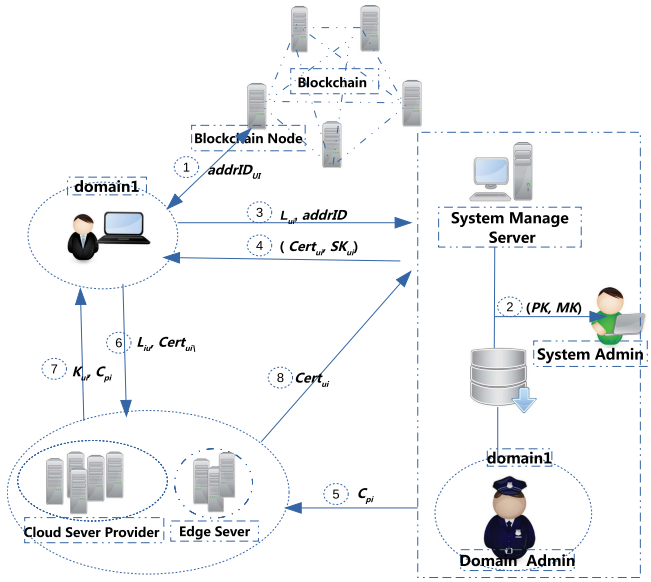


Fig. 2. Implementation process of the smart factory system in IIoT.

5) $\text{Delegate}(\text{PK}, L_{ui}, \text{DK}_{ui}, \text{CT})\to(K_{ui}, \text{CT}')$: Inputs PK, L_{ui}, DK_{ui}, and CT, and returns a decryption parameter K_{ui} and a new ciphertext CT'.
6) $\text{Decrypt}(C'_{pi}, K_{ui}, \text{SK}_{ui})\to m$: Inputs K_{ui}, CT', and SK_{ui}, and outputs the plaintext m.
7) $\text{Trace}(\text{PK}, L_{ui}, \text{DK}_{ui})\to(\text{ID}_{ui}/\perp)$: Inputs PK, L_{ui}, and SK_{ui}, and returns the user's ID_{ui} or $\perp$.
8) $\text{Re-Encrypt}(\text{PK}, (\rho, M), \text{REnc_key})\to\text{CT}'$: Inputs PK, (ρ, M), and REnc_key and CT'.
9) $\text{UpdateUser}(\text{MK}, L_{ui}, \text{ID}_{ui})\to(\text{SK}_{ui})$: Inputs MK, the attributes set L_{ui}, and user identification ID_{ui}, and returns a private secret key SK_{ui}.
10) $\text{CloudUpdateUser}(\text{PK}, L_{ui}, \text{ID}_{ui}, \text{TransformSK})\to(\text{SK}_{ui})$: Inputs PK, L_{ui}, ID_{ui}, and Transform key for update user's delegate key TranksformSK and returns SK_{ui}.
11) $\text{Transaction_save}(K_{\text{pri}}, \text{userAddr}, \text{content}, \text{timeSta-mp})$: The function is that the user stores the information on the blockchain. K_{pri} indicates the signature private key, userAddr indicates the sender of the transaction, content indicates the storage content, and timeStamp indicates the timestamp, the timestamp is provided by the function getTime(). If the storage was successful, it returns true; otherwise, it returns false.

As shown in Fig. 2, the implementation process of the smart factory system as follows.

① The user registers blockchain and applies for account address as an identity ID called addrID_{ui}.

② SMS selects λ and uses $\text{setup}(\lambda)$ to generate PK and MK according to system attributes set L_{sys}, and then send PK and L_{sys} to blockchain.

③ The user sends L_{ui} and ID_{ui} to the SMS to register.

④ SMS uses KeyGen() to generate $(\text{Cert}_{ui}, \text{SK}_{ui})$ and assign it to the user according to the user's L_{ui}, and then send Cert_{ui} to blockchain.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

⑤ The domain manager gets PK and R from blockchain and formulates a corresponding access policy (ρ, M), then constructs an attribute matrix M, executes Encrypt() to generate the ciphertext CT, if the data are temporary, then stores it to ES, otherwise sends it to CSP, and sends (ρ, M) to blockchain.

⑥ The user requests to query the domain data and sends Cert_{ui} and L_{ui} to ES or CSP.

⑦ ES or CSP gets PK form blockchain, and matches policies, if successful, it returns the decryption parameter to user, and user can obtain the plaintext using Decrypt().

⑧ CSP sends Cert_{ui} to SMS, and SMS performs Trace() to track the user. When a malicious user leaks data and the system traces the user, the user's addrID will be added to the revocation list, and the revocation list will be sent to blockchain. ES/CSP re-encrypts the data.

B. Protocol Description

We proposed a scheme that is composed of nine phases: initialization, system setup, key generation, data encryption, data decryption, decryption, malicious user tracking, data re-encryption, user updates private key, and CSP or ES updates the storage private key of the user.

Phase 1. Initialization: It is to complete the user registration on blockchain. First, user registers on blockchain by setting the account password passwrd on the user side to obtain the account public and private key pair (pubKey, priKey). Then, user obtains the account address called addrID_{ui} generated according to pubKey, and publishes addrID_{ui} to blockchain, where $\text{addrID}_{ui} = \text{IDGen(password)}$ and IDGen() is to generate an account address based on the user's account password.

Phase 2. System setup: $\text{setup}(\lambda) \rightarrow (\text{PK}, \text{MK})$: Generates MK and PK. Choose two bilinear groups (G, G_T) of prime order P and define a bilinear map e: $G \times G \rightarrow G_T$, where g is the generators. Set $U = \{\text{addrID}_{ui} | 1 \leq i \leq n\}$ as the user set. The query list W with the initialization state $\emptyset$ is composed of two items (random parameter and addrID). Pick a hash function H: $\{0,1\} \rightarrow G$, H': $G_T \rightarrow Z_p^*$, choose two random numbers $\alpha, \beta \in Z_P$, Then, the system outputs MK and PK, where $\text{MK} = \{\alpha, \beta\}$ and $\text{PK} = \{G, G_T, g, e(g,g)^{\alpha}, g^{\beta}, H(), H'()\}$. MK is kept in secret by the system administrator. PK is sent to blockchain by calling Transaciton_save().

$\text{DomainSetup}(\lambda) \rightarrow (\text{DPK}, \text{DMK})$: Generate a public key for each domain administrator. Randomly select $g_l \in G$ for each domain, the initial version number $d_{l,V} \in Z_p^*$ of the DMK, which is kept in secret by the domain administrator DO_l and computer $e(g,g)^{d_{l,V}}$, and $g^{d_{l,V}}$. Then the system outputs DPK and DMK, where $\text{DPK} = \{\forall i \in [1 \ldots l] \{\text{version}, \text{DO}_l, g_l, e(g,g)^{d_{l,ver}}, g^{d_{l,ver}}\}\}$ and $\text{DMK} = \{d_{l,\text{ver}}\}$. DMK is kept in secret by the domain administrator, and DPK is sent to blockchain too.

Phase 3. Key generation: $\text{KeyGen}(\text{MK}, L_{ui}, \text{addrID}_{ui}) \rightarrow (\text{Cert}_{ui}, \text{SK}_{ui})$: When each user registers, the system chooses user's L_{ui} and addrID_{ui} as its input parameters. It outputs SK_{ui} and Cert_{ui}. Specifically, a user probably in multiple domains, so he can get the corresponding domain key. Randomly choose $r, r_j, r_d \in Z_p^*$,

$$\begin{aligned}
\text{SK}_{ui} = \{&\text{version}, \text{DSK}_{ui} = \{K = g^{\frac{\alpha+r}{\beta}} \quad \forall j \in L_{ui} : \\
&D_{j1} = g^{r_j}, D_{j2} = H(\text{ID}_{ui})^{r d_{l,\text{ver}}} H(\text{attr}_j)^{r_j}\} \\
&\text{DSK}_{\text{GR}} = \{D_{\text{GR}}^1 = g^{\frac{d_{l,\text{ver}}}{\beta}} \cdot H(\text{ID}_{ui})^{\frac{r d_{l,\text{ver}}}{\beta}} \\
&D_{\text{GR}}^2 = g^r \cdot g_l^{r_d}, D_{\text{GR}}^3 = g^{r_d}\}\}. \qquad (1)
\end{aligned}$$

Each user selects an identifier $g_{ui} \in G$, $\text{Cert}_{ui} = \{H(\text{ID}_{ui})^{\beta^2 d_{l,\text{ver}}} g_{ui}^{d_{l,\text{ver}}}, g_{ui}\}$, SK_{ui} will send to user in a secure channel. Cert_{ui} will be public and posted to blockchain. In order to reduce the computation of decryption, the System Manager will delegate SK_{ui} to CSP or ES, random select element $z \in Z_p^*$, and the TK_{ui}, TSK'_{GR} will send to user in a secure channel.

$$\begin{aligned}
\text{TK}_{ui} = \{&\text{TSK}'_{ui} = \{K' = g^{\frac{z(\alpha+r)}{\beta}} \quad \forall j \in L_{ui} : D'_{j1} = g^{z r_j} \\
&D'_{j2} = H(\text{ID}_{ui})^{z r d_{l,\text{ver}}} H(\text{attr}_j)^{z r_j}\} \\
&\text{TSK}'_{\text{GR}} = \{T_{\text{GR}}^1 = g^{\frac{z d_{l,\text{ver}}}{\beta}} \cdot H(\text{ID}_{ui})^{\frac{z r * d_{l,\text{ver}}}{\beta}} \\
&T_{\text{GR}}^2 = g^{zr} \cdot g_l^{z r_d}, T_{\text{GR}}^3 = g^{z r_d}\}\}. \qquad (2)
\end{aligned}$$

Phase 4. Data encryption: $\text{Encrypt}(\text{PK}, (\rho, M), m) \rightarrow C_{pi}$: According to the access strategy of domain, a $l \times n$ weighted access control matrix M is constructed, which l represents the number of attributes. Randomly select vector $\vec{v} = \{s_1, v_2, v_3, \ldots, v_n\} \in Z_p^*$ is used to the distribute secret value s. Calculate $\lambda_i = \vec{v} * M_i$, M_i as the row vector corresponding to the matrix attribute:

$$\begin{aligned}
C_{pi} = \{&\text{Version} = \text{ver}, M, C = m * e(g,g)^{\alpha s_2} e(g,g)^{d_{l,\text{ver}} s_1} \\
&C_1 = g^{\beta s_1}, C_2 = g^{\beta s_2}, C' = g^{s_1}, C'_{\text{GR}} = g^{s_2}, C_{\text{GR}} = g_l^{s_2} \\
&V = g^{H'(e(g,g)^{\alpha s_2} e(g,g)^{d_{l,\text{ver}} s_1})} \\
&\forall y \in l : C_i = g^{\lambda_i}, C'_i = H(\text{attr}_i)^{\lambda_i}\}. \qquad (3)
\end{aligned}$$

Then, the system manager sends the summary of C_{pi} to the ES if the C_{pi} is temporary or recently used, otherwise to the CSP.

Phase 5. Data decryption: ES or CSP help users with many complex calculations with TK_{ui}, First, decrypt the ciphertext item of attribute:

$$\begin{aligned}
A_1 &= \prod_{i \in I} \left(\frac{e(C_i, D'_{j2})}{e(C'_i, D'_{j1})} \right)^{\omega_i} \\
&= \prod_{i \in I} \frac{e(H(\text{attr}_i), g)^{z r_j \lambda_i \omega_i} e(H(\text{ID}_{ui}), g)^{z r d_{l,\text{ver}} \lambda_i \omega_i}}{e(H(\text{attr}_i), g)^{z r_j \lambda_i \omega_i}} \\
&= e(H(\text{ID}_{ui}), g)^{z r d_{l,\text{ver}} s_1}. \qquad (4)
\end{aligned}$$

Then, decrypt the ciphertext item of domain, if one with multiple domains key, he/she just chooses the corresponding domain key, he/she will get A_2, then get A_5:

$$A_2 = \frac{e(T_{\text{GR}}^2, C'_{\text{GR}})}{e(C_{\text{GR}}, T_{\text{GR}}^3)} = \frac{e(g, g_l)^{z t s_2} e(g,g)^{z r s_2}}{e(g, g_l)^{z t s_2}} = e(g,g)^{z r s_2}$$

$$A_3 = e(C_2, K) = e(g^{\beta s_2}, g^{\frac{(\alpha+r)}{\beta}}) = e(g,g)^{z(\alpha+r)s_2}$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

$$A_4 = e(C_1, T^1_{\mathrm{GR}}) = e(g,g)^{zd_{l,\mathrm{ver}}s_1} e(H(ID_{ui}), g)^{zrd_{l,\mathrm{ver}}s_1}$$
$$A_5 = \frac{A_3 * A_4}{A_1 * A_2} = e(g,g)^{z\alpha s_2} e(g,g)^{zd_{l,\mathrm{ver}}s_1} \quad (5)$$

if $V = g^{A_5^{(1/z)}}$, cloud decryption is correct; otherwise, it is wrong, output $\perp$

$$\frac{C}{A_5^{(1/z)}} = \frac{m * e(g,g)^{\alpha s_2} e(g,g)^{d_{l,\mathrm{ver}}s_1}}{(e(g,g)^{z\alpha s_2} e(g,g)^{zd_{l,\mathrm{ver}}s_1})^{1/z}} = m. \quad (6)$$

The decryption at this phase is outsourced. The user's TransformSK is stored in the cloud server, which will be partially decrypted by TransformSK and sent to the user. The user only needs to do simple calculations to complete the final decryption.

Phase 6. Malicious users tracking: Trace(PK, L_{ui}, Cert_{ui}) $\rightarrow$ addrID_{ui}: If a compromised user leads to data leakage, the malicious user must be tracked and added to the revocation list R. When the user requests data, CSP or ES matches policies and send Cert_{ui} to the System Manager. The System Manager needs to verify the validity of Cert_{ui}. If Cert_{ui} is a valid parameter, the corresponding addrID_{ui} can be found in list W. Otherwise, the user has no permission to decrypt the message, and then there is no need to execute tracking.

Verifyphase: SMS verifies the validity of Cert_{ui}. SMS inputs parameters that include L_{ui}, PK and Cert_{ui}. Then checks whether the equation $e(H(\mathrm{addrID}_{ui})^{\beta d_{l,\mathrm{ver}}} g_{ui}{}^{\beta d_{l,\mathrm{ver}}}, g) = e(H(\mathrm{addrID}_{ui}) g_{ui},\ g^{\beta d_{l,\mathrm{ver}}})$ holds, if it holds, the user with addrID_{ui} has leaked the private key.

Phase 7. Data re-encryption: When data are leaked, the malicious data can be queried using the data header, and the addrID_{ui} will be added to the revocation list, the data should be re-encrypted. When a user's private key is leaked, the user's identity is first found, and then the domain's ciphertext is re-encrypted. The system manager randomly choose $d_{l,\mathrm{ver}+1} \in Z_p^*$, calculate $g^{d_{l,\mathrm{ver}+1}}$, $e(g,g)^{d_{l,\mathrm{ver}+1}}$, and make DO_l, $e(g,g)^{d_{l,\mathrm{ver}+1}}$, $g^{d_{l,\mathrm{ver}+1}}$ public. Generate re-encryption key $\mathrm{REnc_key} = g^{\frac{(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}{\beta}}$, and send it to the CSP or ES for ciphertext update.

$$\mathrm{CT} = \{\mathrm{Version} = \mathrm{ver}, M, C = m * e(g,g)^{\alpha s_2} e(g,g)^{d_{l,\mathrm{ver}}s_1},$$
$$C_1 = g^{\beta s_1}, C_2 = g^{\beta s_2}, C' = g^{s_1}, C'_{\mathrm{GR}} = g^{s_2}, C_{\mathrm{GR}} = g_l^{s_2}$$
$$\forall y \in l : C_i = g^{\lambda_i}, C'_i = H(\mathrm{attr}_i)^{\lambda_i}\} \quad (7)$$

$$\mathrm{TransCT} = e(\mathrm{REnc_key}, C_1) = e(g,g)^{s_1(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}$$
$$C_{\mathrm{ver}+1} = C * \mathrm{TransCT} = m * e(g,g)^{\alpha s_2} e(g,g)^{d_{l,\mathrm{ver}+1}s_1}. \quad (8)$$

Phase 8. User updates private key: System manager for each user generates TransSK, and then user updates private key DSK_{ui} and $\mathrm{DSK}_{\mathrm{GR}}$.

$$\mathrm{TransSK} = \left\{ d_1 = H(\mathrm{ID}_{ui})^{r(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})},\right.$$
$$\left. d_2 = g^{\frac{d_{l,\mathrm{ver}}}{\beta}} * H(\mathrm{ID}_{ui})^{\frac{r(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}{\beta}} \right\} \quad (9)$$

$$\mathrm{DSK}_{ui} = \{K = g^{\frac{(\alpha + r)}{\beta}}, L = g^t \quad \forall j \in L_{ui} :$$
$$D_{j1} = g^{r_j}, D_{j2} = H(\mathrm{ID}_{ui})^{d_{l,\mathrm{ver}+1}} H(\mathrm{attr}_j)^{r_j}\},$$
$$\mathrm{DSK}_{\mathrm{GR}} = \{D^1_{\mathrm{GR}} = (g \cdot H(\mathrm{ID}_{ui}))^{\frac{d_{l,\mathrm{ver}+1}}{\beta}},$$
$$D^2_{\mathrm{GR}} = g^r \cdot g_l^{r_d}, D^3_{\mathrm{GR}} = g^{r_d}\}. \quad (10)$$

Phase 9. CSP or ES updates storage private key of user: When received TransCloudSK, CSP or ES update the storage private key of user:

$$\mathrm{TransCloudSK} = \{d_1 = H(\mathrm{ID}_{ui})^{zr(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})},$$
$$d_2 = (g * H(\mathrm{ID}_{ui}))^{\frac{z(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}{\beta}}\}$$
$$D'_{j2} = H(\mathrm{ID}_{ui})^{zrd_{l,\mathrm{ver}}} H(\mathrm{attr}_j)^{zr_j} d_1$$
$$= H(\mathrm{ID}_{ui})^{zrd_{l,\mathrm{ver}+1}} H(\mathrm{attr}_j)^{zr_j}$$
$$T^1_{\mathrm{GR}} = g^{\frac{zd_{l,\mathrm{ver}}}{\beta}} \cdot H(\mathrm{ID}_{ui})^{\frac{zr * d_{l,\mathrm{ver}}}{\beta}} d_2$$
$$= g^{\frac{zd_{l,\mathrm{ver}}}{\beta}} \cdot H(\mathrm{ID}_{ui})^{\frac{zr * d_{l,\mathrm{ver}+1}}{\beta}}. \quad (11)$$

V. Scheme Analysis

A. Security Proof

Security of outsourcing does not need to be proven, because we borrowed the Green scheme, and the security is consistent with it. The following security proof is based on the DBDH assumption and the IND-CPA security model. Suppose the DBDH assumption holds, no polynomial adversary can selectively break the proposed system. If there is an adversary $\mathcal{A}$ who can break this scheme, a polynomial time algorithm $\mathcal{B}$ can be constructed to break the q-DBDH hypothesis. Challenger $\{G, G_T, q, g, e, g^a, g^b\}$ sends to algorithm $\mathcal{B}$ (algorithmB does not know a, b), algorithm $\mathcal{B}$ hopes to output $g^{a/b}$ under the result of adversary $\mathcal{A}$.

Init: $\mathcal{A}$ enters the access control matrix (M^*, ρ^*) to be challenged, Domain to be challenged, and the corresponding version number $d^*_{l,\mathrm{ver}}$.

Setup: $\mathcal{B}$ generate MSK and PK, where $\mathrm{MSK} = \{\alpha, \beta\}$, $\mathrm{PK} = \{G, G_T, g, e(g,g)^\alpha, g^\beta, H(), H'()\}$. $\mathcal{B}$ randomly choose $d_{l,1}, d_{l,2}, \ldots, d_{l,\mathrm{ver}^*} \in Z_p^*$, and calculate $g^{d_{l,\mathrm{ver}}}, e(g,g)^{d_{l,\mathrm{ver}}}$

$$\mathrm{DPK} = \{\mathrm{version}, \mathrm{DO}_l, g_l, e(g,g)^{d_{l,\mathrm{ver}}}, g^{d_{l,\mathrm{ver}}}\}$$
$$\mathrm{DPK}_{\mathrm{ver}^*} = \{\mathrm{version}^*, \mathrm{DO}_l, g_l, e(g, g^b), g^b\}. \quad (12)$$

Phase1:

1) *Random Oracle H Query*

1) Random Oracle query for user identity. $\mathcal{B}$ to build user identity list $L_{\mathrm{addrID}_{ui}}$, if $\mathrm{ID}_{\mathrm{addrui}}$ already in $L_{\mathrm{ID}_{\mathrm{addrui}}}$, i.e., tuple $\langle \mathrm{ID}_{ui}, H_i, l_i, \mathrm{coin} \rangle$ existed, then return $H(\mathrm{addrID}_{ui}) = H_i$ to $\mathcal{A}$.
2) Random Oracle query for user attributes. $\mathcal{B}$ establishes user attribute list L_{attr}, if attr_j already in L_{attr}, i.e., tuple $\langle \mathrm{attr}_j, H_{\mathrm{attr}_j}, a_j \rangle$ existed, H_{attr_j} is used to reply the query of attr_j; otherwise, randomly choose $a_j \in Z_p^*$, $H_{\mathrm{attr}_j} = g^{\mathrm{attr}_j}$, insert tuple $\langle \mathrm{attr}_j, H_{\mathrm{attr}_j}, a_j \rangle$ into list and return H_{attr_j} to $\mathcal{A}$.

2) Type-I Adversary Query. KeyGen Query(): $\mathcal{B}$ establishes user private key list $L^{\text{type-I}}_{\mathrm{sk}}$, if private key of ID_{ui} already in $L^{\text{type-II}}_{\mathrm{sk}}$ then output False and terminate the program. Otherwise,

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

run KeyGen(MK, L_{ui}, addrID$_{ui}$) $\rightarrow$ (Cert$_{ui}$, SK$_{ui}$), where the set of attribute L_{ui} satisfies the access control matrix (M^*, ρ^*), The addrID$_{ui}$ current version number version $<$ version*

$$\mathrm{SK}_{ui}^{I} = \mathrm{DSK}_{ui}^{I} = \{K = g^{(a+r)/\beta}, L = g^{t} \quad \forall j \in L_{ui} : D_j = ((g^a)^{l_i})^{d_{l,\mathrm{ver}}} H_{\mathrm{attr}_j}^{t}\} \quad (13)$$

$$\mathrm{DSK}_{\mathrm{GR}}^{\mathrm{I}} = \{D_{\mathrm{GR}}^{1} = (g \cdot (g^a)^{l_i})^{d_{l,\mathrm{ver}}/\beta}, D_{\mathrm{GR}}^{2} = g^{r} \cdot g_l^{t}\} \quad (14)$$

write $\langle \mathrm{addrID}_{ui}, \mathrm{SK}_{ui}^{\mathrm{I}} \rangle$ to $L_{\mathrm{sk}}^{\text{type-I}}$ and return SK$_{ui}$ to $\mathcal{A}$.

3) Type-II Adversary Query. KeyGenQuery(): If private key of addrID$_{ui}$ appears as $\langle \mathrm{addrID}_{ui}, \mathrm{SK}_{ui}^{\mathrm{I}} \rangle$ in $L_{\mathrm{sk}}^{\text{type-II}}$, then output False and terminate the program. If private key of addrID$_{ui}$ appears as $\langle \mathrm{addrID}_{ui}, \mathrm{SK}_{ui}^{II} \rangle$ in $L_{\mathrm{sk}}^{\text{type-II}}$ then send SK$_{ui}^{\mathrm{II}}$ to $\mathcal{A}$. Otherwise, algorithm $\mathcal{B}$ runs KeyGen(MK, L_{ui}, addrID$_{ui}$) $\rightarrow$ (Cert$_{ui}$, SK$_{ui}$), where the set of attribute L_{ui} does not satisfy access control matrix (M^*, ρ^*), the current version number of user ID$_{ui}$ is version $=$ version*

$$\mathrm{SK}_{ui}^{\mathrm{II}} = \mathrm{DSK}_{ui}^{\mathrm{II}} = \{K = g^{(a+r)/\beta}, L = g^{t} \quad \forall j \in L_{ui} : D_{j1} = g^{r_j}, D_{j2} = (g^b)^{rl_i} H_{\mathrm{attr}_j}^{t}\} \quad (15)$$

$$\mathrm{DSK}_{\mathrm{GR}}^{\mathrm{II}} = \{D_{\mathrm{GR}}^{1} = (g \cdot (g^b)^{l_i})^{1/\beta}, D_{\mathrm{GR}}^{2} = g^{r} \cdot g_l^{r_d}, D_{\mathrm{GR}}^{3} = g^{r_d}\} \quad (16)$$

write $\langle \mathrm{addrID}_{ui}, \mathrm{SK}_{ui}^{\mathrm{II}} \rangle$ to $L_{sk}^{\text{type-II}}$ and return SK$_{ui}$ to $\mathcal{A}$.

4) Re-Encrypt Query.

When the Version flag of CT$_{\mathrm{ver}}$ is less than the current Version*, proxy re-encryption is performed. First $\mathcal{B}$ generates the re-encryption key REnc $= g^{(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})/\beta}$ and updates the ciphertext to CT$_{\mathrm{ver}^*}$. Then generates and sends TransSK $= \{d_1 = ((g^a)^{l_i})^{r(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}, \; d_2 = g^{(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})/\beta} * ((g^a)^{l_i})^{r(d_{l,\mathrm{ver}+1} - d_{l,\mathrm{ver}})}\}$ to $\mathcal{A}$ to update the private key.

Challenge: $\mathcal{A}$ presents m_0, m_1 that length of the two ciphertexts is equal, and $\mathcal{B}$ chooses $b \in \{0, 1\}$ to encrypt, then

$$\mathrm{CT} = \{\mathrm{Version}^*, M, C = m * e(g,g)^{as_2} e(g,g)^{d_{l,\mathrm{ver}} s_1} \quad C_1 = g^{\beta s_1}, C_2 = g^{\beta s_2}, C' = g^{s_1}, C'_{\mathrm{GR}} = g^{s_2} \quad C_{\mathrm{GR}} = g_l^{s_2}, V = g^{H'(e(g,g)^{as_2} e(g,g)^{d_{l,\mathrm{ver}} s_1})} \quad \forall y \in l : C_i = g^{\lambda_i}, C'_i = H(\mathrm{attr}_i)^{\lambda_i}\}. \quad (17)$$

Phase2: It is the same as Phase1.

Guess: Finally, $\mathcal{B}$ ignores $\mathcal{A}$s output and randomly selects a DSK$_{I,i}$ (as Type-I query's response) and a DSK$_{\mathrm{II},i}$ (as Type-II query's response). Theoretically, if $\mathcal{A}$ can decrypt the challenge ciphertext CT$_{\tilde{b}}$ only when he has T_{GR}^{1} of SK$_{\mathrm{II},i}$ matches D_j of SK$_{\mathrm{I},i}$. We denote $\langle \mathrm{ID}'_i, H'_i, l'_i, \mathrm{coin} = 0 \rangle$ corresponds to SK$_{I,i}$ and $\langle \mathrm{ID}'_j, H'_i, l'_i, \mathrm{coin} = 0 \rangle$ corresponds to SK$_{\mathrm{II},i}$. We formulate abovementioned theory evidence as $(g^b)^{l'_i} = (H_i)^{r d_{l,\mathrm{ver}}}$, which means $g^{b \cdot l'_i r \cdot d_{l,\mathrm{ver}}} = g^{a \cdot l_i \cdot d_{l,\mathrm{ver}}}$. So, the outputs $g^{a/b} = g^{l'_i / l_i}$ as $\mathcal{B}$s answer. If $\mathcal{B}$ returns is not $\perp$ in phase1, which means $\mathcal{A}$s view just like really attack. Then, in simulation, it calculates $\mathcal{B}$s likelihood of success, completing the proof. Assuming $\mathcal{A}$ makes q_{I} Type-I query and q_{II} Type-II query. $\mathcal{B}$s probability returns $\delta^{q_{\mathrm{I}}} \cdot (1 - \delta)^{q_{\mathrm{II}}}$ (simplify, assuming $q_{\mathrm{I}} = q_{\mathrm{II}}$) instead of $\perp$ in Phases 1. Moreover, when $\delta = 1/2$, getting the maximum value. Hence, $\mathcal{B}$s succeeds probability during the simulation is $1/2(q_{\mathrm{I}} + q_{\mathrm{II}})$. $\mathcal{B}$s probability selects the correct SK$_{\mathrm{I},i}$ and SK$_{\mathrm{II},i}$ is $1/q_{\mathrm{I}} \cdot q_{\mathrm{II}}$. It shows that the advantage of $\mathcal{B}$ is at maximum $\varepsilon/2^{(q_{\mathrm{I}} + q_{\mathrm{II}})}$.

B. Comparing

In this section, we compare the security properties of our scheme with those of other schemes. To this end, we have chosen the recently proposed attribute-based encryption data sharing protocols [22], [23] as a benchmark. The specific comparison results are described in Table III.

The common point of all five schemes is that all of them have been proven to be secure under the IND-CPA model. Both [25] and our scheme use access trees to achieve policy matching, which provides better flexibility compared to [22], [13], and [23]. Among the five schemes, only [13] implements policy hiding. In terms of user revocation, [13], [22] implement indirect revocation, while [23], [25], and our scheme provide direct revocation. In addition, only our scheme can provide user tracking.

VI. Simulation Experiment

We evaluate the performance in this section and the essential parameters are listed in Table IV. For this simulation experiment, we run 100 times linear pair and exponential operation on a machine, where operating system is Windows 8 (64-bit, Intel Core i5 CPU, @2.4 GHz and 4 G RAM).

A. Communication Overhead

The communication overhead is evaluated by comparing our scheme with [13], [22], [23], [25] regarding the curves of encryption time, decryption time, and public/private generation time with the number of attributes. The details are shown in Fig. 3.

First, we compare the curves of the encryption time with the number of attributes for the five schemes (Example [13], [22], [23], [25], and ours). Fig. 3(a) shows that the encryption time of all five schemes increases linearly as the number of attributes increases from 5 to 30 when we set the parameter r is 1. Among them, [13] grows the fastest and requires the longest encryption time. The other four schemes (see [22], [23], [25], and ours) grow at almost the same rate, but our scheme has the shortest encryption time. Fig. 3(d) depicts the curves of the encryption time with the number of attributes for the five schemes when we set the parameter r is 3. Fig. 3(d) is basically the same as Fig. 3(a), the only difference is that the encryption time of our scheme is slightly higher than [22], [23], and [25].

Next, we compare the curves of the decryption time with the number of attributes for the five schemes (Example [13], [22], [23], [25], and ours). As shown in Fig. 3(b), the decryption time for [13], [23], and [25] increases gradually as the number of attributes increases from 5 to 30 when $r = 1$, but [22] and ours remain almost unchanged. Fig. 3(e) depicts the curves of the decryption time with the number of attributes for the five schemes when we set the parameter r is 3. Fig. 3(e) is basically

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISONS OF SECURITY PROPERTIES BETWEEN SCHEMES

References	Direct user revocation	Indirect user revocation	Traceability	Policy hiding	Policy matching	Proved security in IND-CPA
[13]	×	√	×	√	AND gate	√
[22]	×	√	×	×	LSSS	√
[23]	√	×	×	×	LSSS	√
[25]	√	×	×	×	access tree	√
The proposed model	√	×	√	×	access tree	√

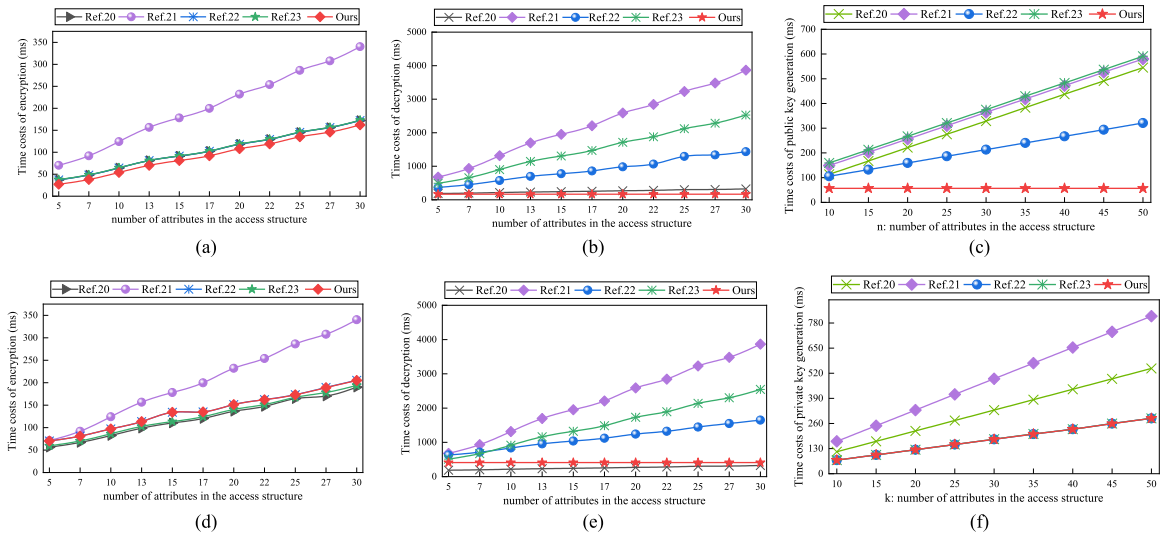


Fig. 3. Time costs of encryption/decryption, public/private key generation time with the number of attributes in the access structure r. (a) Time costs of encryption when $r = 1$. (b) Time costs of decryption when $r = 1$. (c) Time costs of public key generation. (d) Time costs of encryption when $r = 3$. (e) Time costs of decryption when $r = 3$. (f) Time costs of private key generation.

TABLE IV
SIMULATION EXPERIMENT PARAMETERS

Parameter	Values	Parameter	Values	Parameter	Values
n	$10 \sim 50$	r	$0 \sim 5$	k	$10 \sim 50$
$\|w\|$	$5 \sim 30$	$\|G\|$	161 bits	$\|G_r\|$	161 bits
Z_p	160 bits	t_b	40.7 ms	t_e	5.4 ms

the same as Fig. 3(b), the only difference is that the decryption time of our scheme is slightly higher than [22].

Finally, we compare the curves of the public/private key generation time with the number of attributes for the five schemes (Example [13], [22], [23], [25], and ours). Fig. 3(c) shows the curve of the public key generation time with the number of attributes. As shown in this figure, as the number of attributes increases from 10 to 50, the public key generation time for [22], [13], [23], and [25] grows linearly. Among them, [23] and [13] grow the fastest, followed by [22] and [25]. Moreover, the public key generation time of our scheme remains almost unchanged. Fig. 3(f) shows the curve of the private key generation time with the number of attributes. The private secret generation time for all five schemes increases linearly with the number of attributes. Among them, [13] grows the fastest, followed by [22], and the remaining three schemes grow the slowest and maintain roughly the same growth rate.

B. Storage Overhead

The storage overhead is evaluated by comparing our scheme with [22], [13], [23], and [25] regarding the curves of the length of public key/private key/ciphertext with the number of attributes as well as the length of ciphertext with the number of users in the revocation set. The details are shown in Figs. 4 and 5.

As shown in Fig. 4(a), with the increase of number of attributes from 10 to 50, the storage overhead of the public key in [13], [22], [23], and [25] increases linearly, but our scheme keeps constant and is smaller than [13], [22], and [23] where the number of attributes is over 25, and it is the smallest after the number of attributes over 45. As shown in Fig. 4(b) with the growth of number of attributes from 10 to 50, the storage overhead of the private key in all five schemes increases linearly. However, our scheme has the slowest growth. Furthermore, as shown in Figs. 4(c) and 5, with the increase of number of attributes, the ciphertext length in our scheme is approximately equal to [22], [23], and [25] but much smaller than [13]. With the increase of the number of users in the revocation set, the length of the ciphertext becomes slightly higher than that of [22], [23], and [25], and much smaller than [13].

C. Throughput of Blockchain

We chose the Ethereum private chain as the platform for the blockchain used in the construction plan. The operating environment is Ubuntu Linux 18.04LTS, Intel core8565 u CPU × 4 1.80 GHz, and the memory is 8 GB. Throughput has always been an important indicator for evaluating network online systems. The so-called blockchain throughput is the number of transactions (accesses) that can be processed per second. In this

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

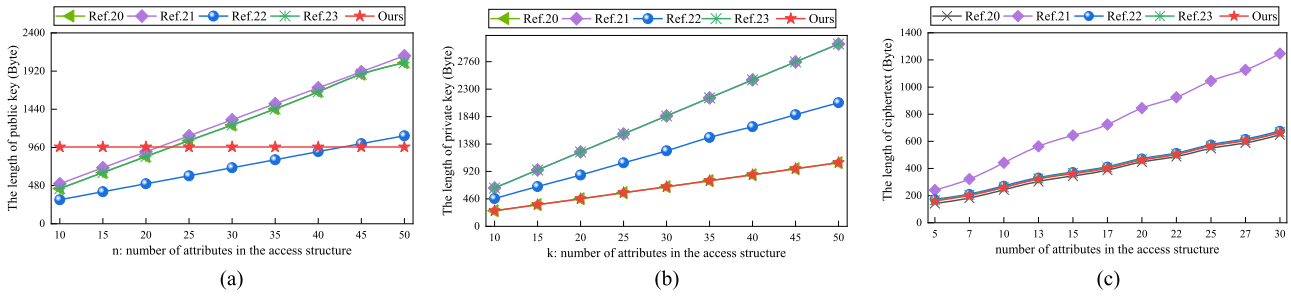


Fig. 4. Storage overhead of public key/private key/ciphertext with the number of attributes. (a) Storage overhead of public key. (b) Storage overhead of private key. (c) Storage overhead of ciphertext.

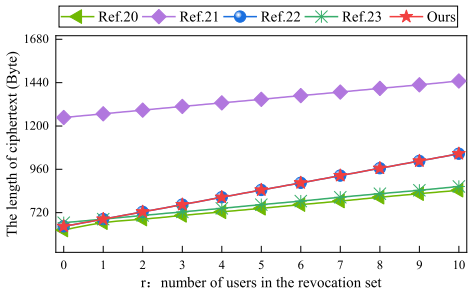


Fig. 5. Storage overhead of ciphertext with the number of users in the revocation set.

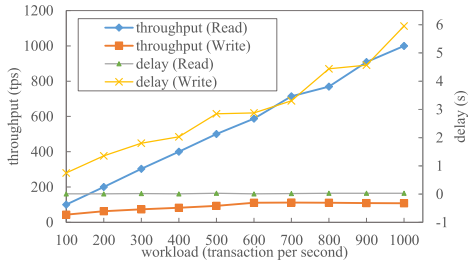


Fig. 6. Throughput and delay for blockchain READ and WRITE operations.

experiment, we test two situations of access to the blockchain in the smart factory. One is that a large number of applications in the smart factory read data from the blockchain; the second is that a large number of applications in the smart factory have transactions with the blockchain, and write data to the blockchain. The number of transactions per second used in the experiments is from 100 to 1000, and the experimental result is the average number of 10 runs. Fig. 6 shows the performance of throughput and delay for blockchain READ and WRITE operations. When performing READ operations in blockchain, our system can provide high throughput and low latency. Furthermore, our scheme works good in Ethereum private chain and the throughput and delay are acceptable when performing WRITE operations in blockchain.

VII. Conclusion

In the IIoT, secure storage and sharing of smart factory data in the cloud was quite a challenge for different systems and users. In this article, we proposed an enhanced security access control scheme that supported user identity-based traceability and revocation. In our scheme, a unique identity parameter was generated and embedded in the private key generation phase to identify and resist collusion attacks. The decryption key can be calculated by a user with the attributes that satisfy the access policy and who was not in the revocation list. Malicious users can then be tracked according to the tracking list and revoked directly. Future research potential includes partial rather than full revocation for different users' leak behavior, and implementation of fine-grained restrictions for the direct revocation mode.

References

[1] O. Bouachir, M. Aloqaily, L. Tseng, and A. Boukerche, "Blockchain and fog computing for cyberphysical systems: The case of smart industry," *Computer*, vol. 53, no. 9, pp. 36–45, Sep. 2020.

[2] M. Saturno, V. M. Pertel, F. Deschamps, and E. de FR Loures, "Proposal of an automation solutions architecture for Industry 4.0," in *Proc. 24th Int. Conf. Prod. Res.*, Poznan, Poland, 2017, pp. 1–6.

[3] I. Al Ridhawi, Y. Kotb, M. Aloqaily, Y. Jararweh, and T. Baker, "A profitable and energy-efficient cooperative fog solution for IoT services," *IEEE Trans. Ind. Informat.*, vol. 16, no. 5, pp. 3578–3586, May 2020.

[4] V. Alcácer and V. Cruz-Machado, "Scanning the Industry 4.0: A literature review on technologies for manufacturing systems," *Eng. Sci. Technol., Int. J.*, vol. 22, no. 3, pp. 899–919, 2019.

[5] H. Nguyen, K. Tran, X. Zeng, L. Koehl, P. Castagliola, and P. Bruniaux, "Industrial Internet of Things, big data, and artificial intelligence in the smart factory: A survey and perspective," in *Proc. Int. Conf. Data Sci. Bus., Finance Ind.*, 2019, pp. 72–76.

[6] J. Wan, J. Li, M. Imran, D. Li, and Fazal-e-Amin, "A blockchain-based solution for enhancing security and privacy in smart factory," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3652–3660, Jun. 2019.

[7] B. Chen, J. Wan, L. Shu, P. Li, M. Mukherjee, and B. Yin, "Smart factory of Industry 4.0: Key technologies, application case, and challenges," *IEEE Access*, vol. 6, pp. 6505–6519, 2018.

[8] C. Feng, K. Yu, M. Aloqaily, M. Alazab, Z. Lv, and S. Mumtaz, "Attribute-based encryption with parallel outsourced decryption for edge intelligent IoV," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13784–13795, Nov. 2020.

[9] D. Berdik, S. Otoum, N. Schmidt, D. Porter, and Y. Jararweh, "A survey on blockchain for information systems management and security," *Inf. Process. Manage.*, vol. 58, no. 1, 2021, Art. no. 102397.

[10] L. Cheng *et al.*, "SCTSC: A semicentralized traffic signal control mode with attribute-based blockchain in IoVs," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 6, pp. 1373–1385, Dec. 2019.

[11] J. Huang *et al.*, "Blockchain-based mobile crowd sensing in industrial systems," *IEEE Trans. Ind. Informat.*, vol. 16, no. 10, pp. 6553–6563, Oct. 2020.

[12] I. A. Ridhawi, S. Otoum, M. Aloqaily, and A. Boukerche, "Generalizing AI: Challenges and opportunities for plug and play AI solutions," *IEEE Netw.*, early access, doi: 10.1109/MNET.011.2000371.

[13] J. Li, Y. Shi, and Y. Zhang, "Searchable ciphertext-policy attribute-based encryption with revocation in cloud storage," *Int. J. Commun. Syst.*, vol. 30, no. 1, 2017, Art. no. e2942.

[14] M. Al-Khafajiy *et al.*, "Intelligent control and security of fog resources in healthcare systems via a cognitive fog model," *ACM Trans. Internet Technol.*, 2020. [Online]. Available: https://doi.org/10.1145/3382770

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.

[15] J. Jung, B. Song, K. Watson, and T. Uslander, "Design of smart factory web services based on the industrial Internet of Things," in *Proc. 50th Hawaii Int. Conf. Syst. Sci.*, 2017, pp. 5941–5946.
[16] M. Shin, J. Woo, I. Wane, S. Kim, and H. Yu, "Implementation of security mechanism in IIoT systems," in *Proc. 6th Int. Conf. Green Human Inf. Technol.*, 2019, pp. 183–187.
[17] V. Domova and A. Dagnino, "Towards intelligent alarm management in the age of IIoT," in *Proc. Global Internet Things Summit.*, 2017, pp. 1–5.
[18] S. Huh, S. Cho, and S. Kim, "Managing IoT devices using blockchain platform," in *Proc. 19th Int. Conf. Adv. Commun. Technol.*, 2017, pp. 464–467.
[19] L. Tan, N. Shi, K. Yu, M. Aloqaily, and Y. Jararweh, "A blockchain empowered access control Internet of Things, "*ACM Trans. Internet Technol.*, 2021.
[20] S. Yang, P. Lai, and J. Lin, "Design role-based multi-tenancy access control scheme for cloud services," in *Proc. Int. Symp. Biometrics Secur. Technol.*, 2013, pp. 273–279.
[21] L. Popa, M. Yu, S. Ko, S. Ratnasamy, and I. Stoica, "Cloudpolice: Taking access control out of the network," in *Proc. 9th ACM SIGCOMM Workshop Hot Topics Netw.*, 2010, pp. 1–6.
[22] R. Ostrovsky, A. Sahai, and B. Waters, "Attribute-based encryption with non-monotonic access structures," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 195–203.
[23] C. Bai, Y. Zhang, H. Ma, and Z. Liu, "Expressive ciphertext-policy attribute-based encryption with direct user revocation," *Int. J. Embedded Syst.*, vol. 9, no. 6, pp. 495–504, 2017.
[24] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 321–334.
[25] X. Yan and H. Meng, "Ciphertext policy attribute-based encryption scheme supporting direct revocation," *J. Commun.*, vol. 37, no. 5, pp. 44–50, 2016.
[26] K. T. Nguyen, N. Oualha, and M. Laurent, "Securely outsourcing the ciphertext-policy attribute-based encryption," *World Wide Web*, vol. 21, no. 1, pp. 169–183, 2018.
[27] H. Ma, R. Zhang, Z. Wan, Y. Lu, and S. Lin, "Verifiable and exculpable outsourced attribute-based encryption for access control in cloud computing," *IEEE Trans. Depend. Sec. Comput.*, vol. 14, no. 6, pp. 679–692, Nov.-Dec. 2017.
[28] P. Zhang, Z. Chen, J. K. Liu, K. Liang, and H. Liu, "An efficient access control scheme with outsourcing capability and attribute update for fog computing," *Future Gener. Comput. Syst.*, vol. 78, pp. 753–762, 2018.
[29] J. Huang, L. Kong, G. Chen, M. Wu, X. Liu, and P. Zeng, "Towards secure industrial IoT: Blockchain system with credit-based consensus mechanism," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3680–3689, Jun. 2019.
[30] C. Lee, S. Zhang, and K. Ng, "Development of an industrial Internet of Things suite for smart factory towards re-industrialization," *Adv. Manuf.*, vol. 5, no. 4, pp. 335–343, 2017.
[31] C. H. Liu, Q. Lin, and S. Wen, "Blockchain-enabled data collection and sharing for industrial IoT with deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3516–3526, Jun. 2019.
[32] S. Jang, J. Guejong, J. Jeong, and B. Sangmin, "Fog computing architecture based blockchain for industrial IoT," in *Proc. Int. Conf. Comput. Sci.*, 2019, pp. 593–606.
[33] N. Teslya and I. Ryabchikov, "Blockchain-based platform architecture for industrial IoT," in *Proc. 21st Conf. Open Innovations Assoc.*, 2017, pp. 321–329.
[34] J. Li, W. Yao, Y. Zhang, H. Qian, and J. Han, "Flexible and fine-grained attribute-based data storage in cloud computing," *IEEE Trans. Serv. Comput.*, vol. 10, no. 5, pp. 785–796, Sep.-Oct. 2017.



Keping YU (Member, IEEE) received the M.E. and Ph.D. degrees is global information and telecommunication studies from the Graduate School of Global Information and Telecommunication Studies, Waseda University, Tokyo, Japan, in 2012 and 2016, respectively.

He is currently a Researcher with the Global Information and Telecommunication Institute, Waseda University. He is also a Visiting Professor with the College of Computer Science, Sichuan Normal University, Chengdu, China. His research interests include smart grids, information-centric networking, the Internet of Things, blockchain, and information security.



Liang Tan received the Ph.D. degree in computer science from the University of Electronic Science and Technology of China, Chengdu, China, in 2007.

He is currently a Professor with the College of Computer Science, Sichuan Normal University, Chengdu, China, and a Postdoctoral Research Associate with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. His research interests include cloud computing, big data, trusted computing, and network security.



Moayad Aloqaily (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Ottawa, Ottawa, ON, Canada, in 2016.

He is the Managing Director of xAnalytics Inc., Toronto, ON, Canada, 2019. He is currently with the Faculty of Engineering, Al Ain University, Al Ain, UAE. His current research interests include the applications of AI and ML, connected and autonomous vehicles, blockchain solutions, and sustainable energy and data management.

Dr. Aloqaily has chaired and co-chaired many IEEE conferences and workshops. He has served as a Guest Editor in many journals including IEEE WIRELESS COMMUNICATIONS, IEEE NETWORK, and *Computer Network*. He is also an AE with many journals. He has also been appointed the Co-EiC of IEEE CommSoft TC eLetter, 2020. He is an ACM member and a Professional Engineer Ontario (P.Eng.).



Hekun Yang received the master's degree in computer science and technology from Sichuan Normal University, Chengdu, China, in 2020.

He is currently a Teaching Assistant with the School of Computer Science Department, SiChuan Aerospace Vocational College, Chengdu, China. His research interests include cloud computing and information security.



Yaser Jararweh (Member, IEEE) received the Ph.D. degree in computer engineering from The University of Arizona, Tucson, AZ, USA, in 2010.

He is currently an Assistant Professor of computer science with Duquesne University, Pittsburgh, PA, USA. He has co-authored several technical papers in established journals and conferences in the fields related to cloud computing, edge data analytics, and big data.

Dr. Jararweh was a Steering Committee Member and Co-Chair for the for several IEEE events. He was a Guest Editor for many special issues in different established journals. He is also the Steering Committee Chair of the IBM Cloud Academy Conference. He is an Associate Editor for the *Cluster Computing Journal* (Springer), *Information Processing and Management* (Elsevier), among many other journals.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:09:27 UTC from IEEE Xplore. Restrictions apply.